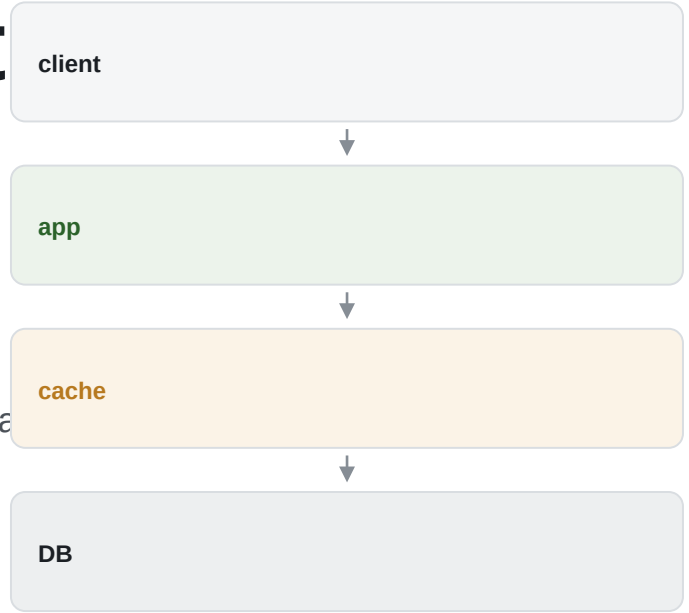
ELEVATE · SESSION 1 · 3 HOURS

Building Scalable Backend Syst

Scaling a single application — from one box to many

Monolith vs microservices · clustering · caching · rate limiting · queues

One story: we start with an app talking straight to a database, and add layers until it can serve thousa users.



each slide adds a layer

AGENDA

What we'll cover in 3 hours

1	Scalable Monolith vs Microservices	30 min
2	Clustering & Horizontal Scaling	30 min
3	Caching Strategy Architecture	35 min
4	Rate Limiting & API Protection	25 min

15 minutes: discussion / break · Total 180 minutes

THE STORY

One app, growing



The whole session is this journey. Each section is a decision point on the way from a script to a platform.

TOPIC 1 · 30 MIN

Scalable Monolith vs Microservices

The shape of the application, and how it changes as it grows.

MONOLITH · STEP 1

Two tiers: app → database



Where this works

One user, one machine, no network. The app writes SQL straight into a local database file.

Desktop and mobile apps often still start exactly like this.

Why two tiers stop working

Exposed database

Clients connect directly to the DB. Credentials live in the app, and any client can run any query.

Duplicated logic

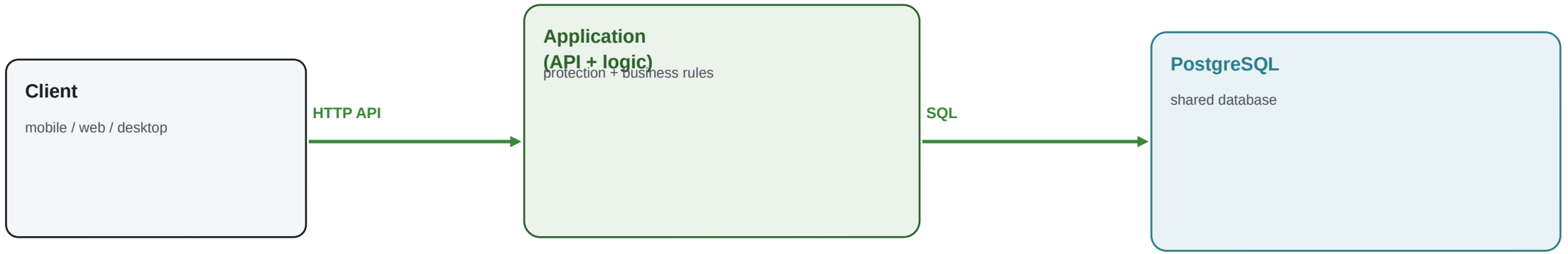
Two apps = two copies of the same rules. Fix a bug in one, the other stays broken.

No single source of truth

Every client has its own view of the data and its own version of the business rules.

The moment you have two clients — or one client that must be protected — you need a layer in the middle.

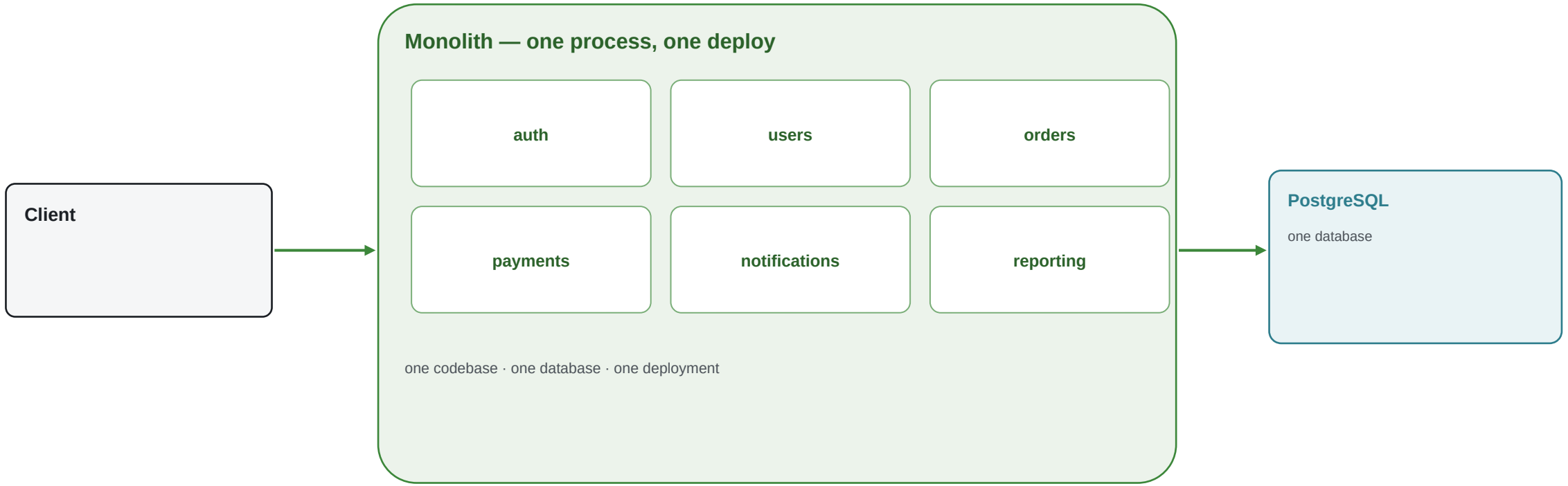
Three tiers: client → API → DB



The middle tier owns the rules. Clients talk to it; only it talks to the database.

- Protection — credentials and queries are no longer exposed to the client.
- Unified business logic — one place to fix a rule, every client gets the fix.

The monolith: one deployable, many modules



The modules are folders in one project — not separate programs. They share one database and one memory space.

Why the monolith is great

Simplicity

One repo, one deploy, one thing to understand. A new hire can run the whole system locally.

Transactions

One database means ACID transactions across features — no distributed transactions.

Performance

Modules call each other in-process. No network hop, no serialization.

Easy operations

One artifact to build, test, deploy and monitor. One log stream.

Most successful products started as monoliths. The architecture is not the problem — the scale is.

Where the monolith hurts

Tight coupling

Everything shares the same code and DB. A change in one module can ripple into others.

Scale the whole thing

If one endpoint is hot, you deploy the whole app. No independent scaling.

One release = one risk

Every deploy ships every feature. A small change can take down everything.

Tech lock-in

The whole app is one language and framework. Adding a better tool for one job is painful.

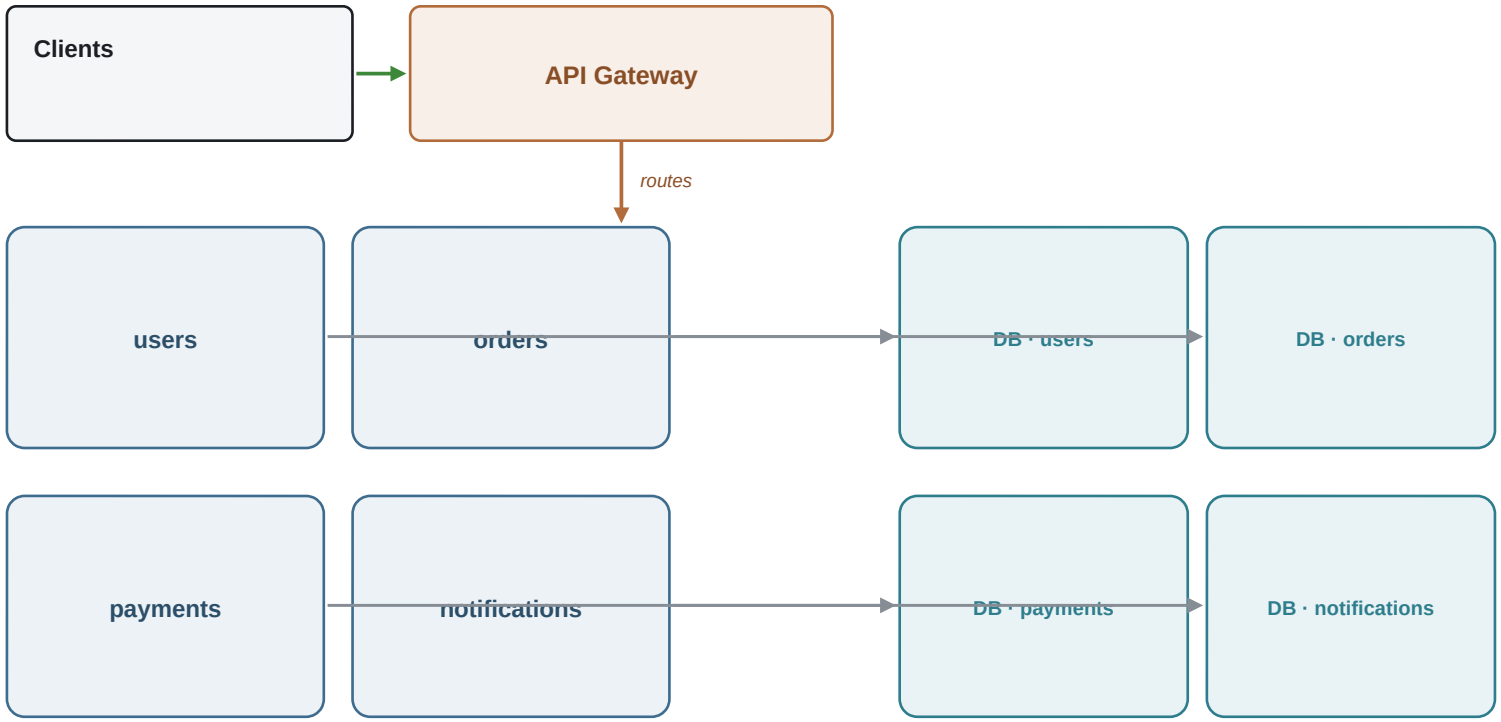
When the monolith cracks

Microservices solve organisational problems first, technical ones second.

Team growth	Ten teams editing one codebase collide. Merge conflicts, no ownership.
Independent scaling	Orders spike on Friday; auth barely moves. You want to scale them separately.
Independent deploys	Each team wants to ship on its own schedule without waiting on the whole app.
Different tech needs	Search wants Elixir, billing wants strict types. One language for everything is a tax.

If none of these apply, stay a monolith. Complexity you do not need is still complexity.

Split by business capability



Each service:

- owns one business capability
- has its own database
- deploys and scales alone
- talks over the network

The monolith's modules become independent deployables behind a gateway.

How services talk to each other

Request / response

HTTP / REST

human-readable, easy to debug

gRPC

binary, typed, low latency

synchronous — caller waits for a reply

Message / event

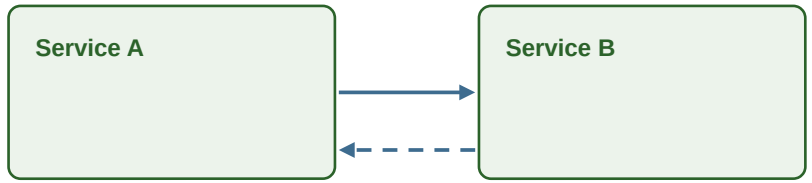
Message brokers

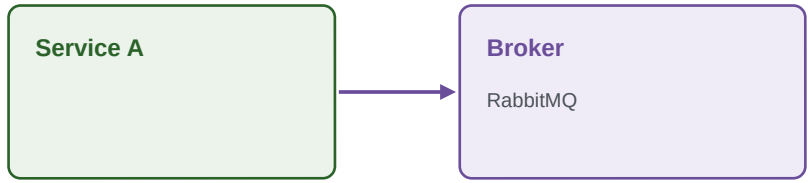
RabbitMQ, Kafka — queued, durable

Event streams

Kafka — ordered log of events

asynchronous — sender keeps going





Rule of thumb: request/response when you need an answer now; message/event when you need to decouple or survive a burst.

MICROSERVICES · TRADE-OFF

Freedom has a price

Benefits

- Independent scaling and deploys
- Team ownership of a service
- Best tool per service
- Fault isolation — one crash stays contained

Costs

- Network calls, latency, failure modes
- Distributed transactions are hard
- Operational complexity (many deploys)
- Observability across many processes

You trade in-process calls for network calls, and one database for many. Those are real, permanent costs.

How to decide

Dimension	Monolith	Microservices
Team size	small (1–2 teams)	many teams
Scale pattern	even, predictable	uneven, per-feature
Deploy cadence	aligned	independent
Tech variety	one language	polyglot
Failure blast radius	whole app	one service
Operational load	low	high

Start monolith. Split when a team or a scaling need forces it — and prefer a modular monolith first.

What to take away

01

Two tiers breaks fast

As soon as you have two clients, you need a middle tier for protection and shared logic.

02

Monolith = one deployable

All modules in one process and one database. Simple, transactional, easy to run.

03

Split on organisation, not fashion

Microservices solve team and scaling problems — at a real technical cost.

04

Transport matters

Request/response when you need an answer; message/event to decouple.

TOPIC 2 · 30 MIN

Clustering & Horizontal Scaling

More machines, more processes — and why statelessness unlocks them.

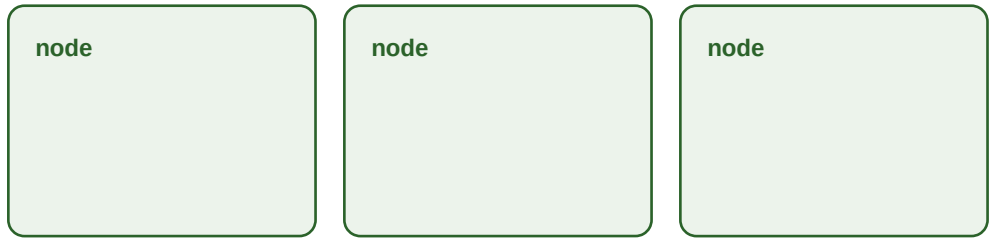
Vertical vs horizontal scaling

Vertical (scale up)



- No code changes
- Hits a hardware ceiling, gets expensive

Horizontal (scale out)



- Add more machines
- Requires statelessness + a load balancer

Horizontal scaling is the point of this whole session — and it only works if the app holds no local state.

Statelessness unlocks scaling

Stateful app (the problem)

node 1

session in memory

node 2

session in memory

User hits node 2 next, but their session lives on node 1 → logged out, or stuck to one node.

Stateless app (the fix)

node 1

no local session

node 2

no local session

Any node can serve any request. Session lives in a JWT the client sends with every call.

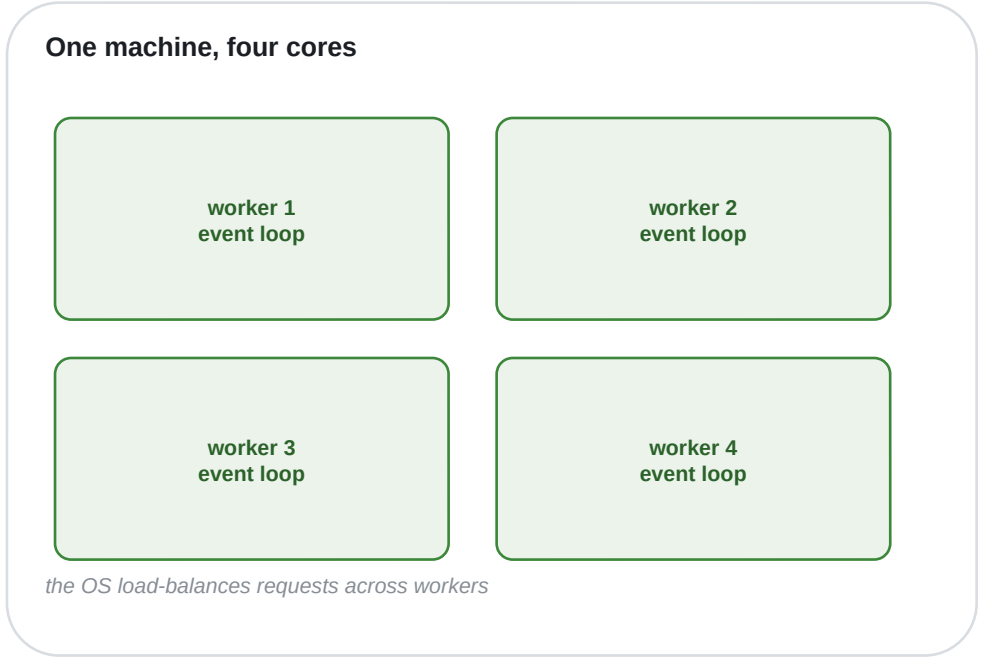
Rule: nothing that must survive a request may live in process memory. Put it in the database, Redis, or a token the client carries.

Node.js cluster: one process per core

cluster.mjs

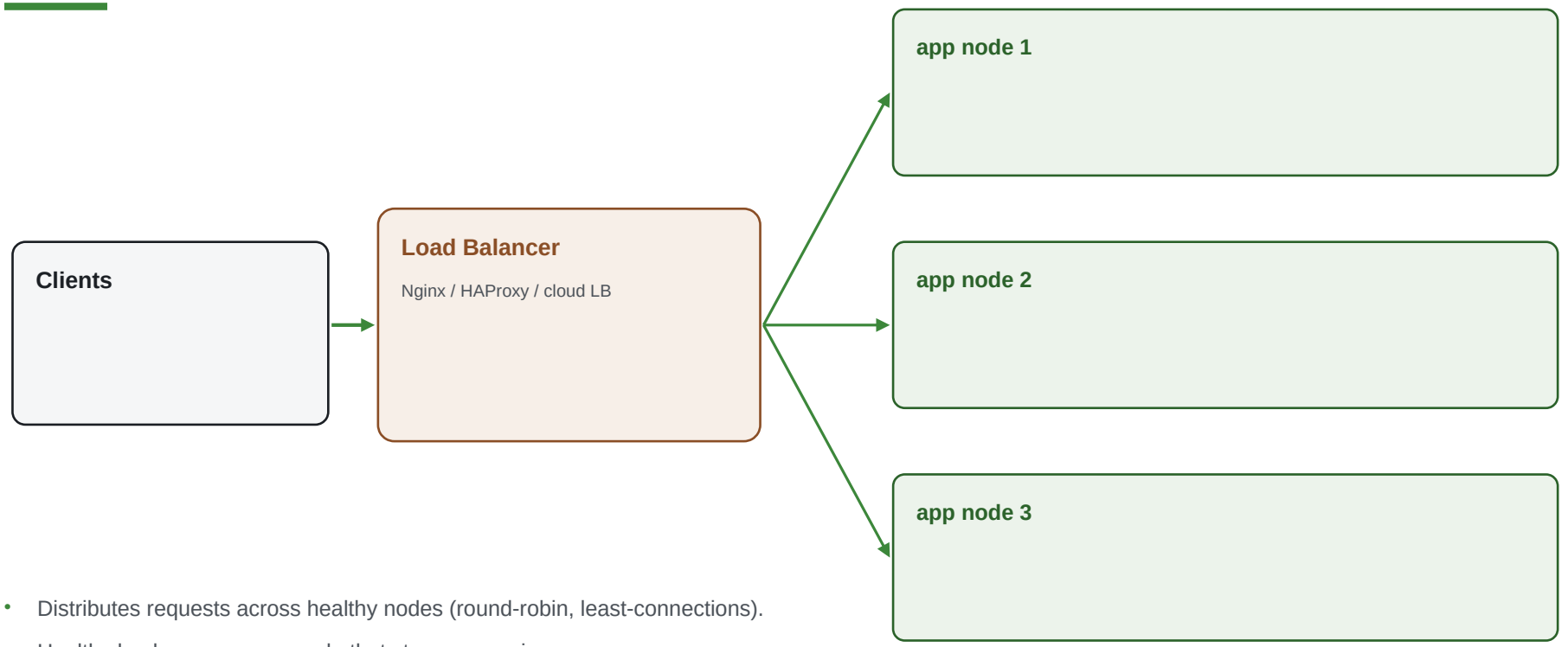
```mjs
import cluster from 'node:cluster';
import { availableParallelism } from 'node:os';

if (cluster.isPrimary) {
  const cpus = availableParallelism();
  for (let i = 0; i < cpus; i++) cluster.fork();
} else {
  // one worker per core, each with its own event loop
  app.listen(3000);
}
```



Node is single-threaded; to use all cores of one machine, run one event loop per core.

The load balancer



- Distributes requests across healthy nodes (round-robin, least-connections).
- Health checks: removes a node that stops answering.
- One public address; the pool of nodes hides behind it.

Horizontal scaling, end to end



- Scale the app by adding nodes; the balancer spreads the load.
- Stateless app + JWT = any node can serve any user.
- Eventually the database becomes the bottleneck — add read replicas.

Stateless auth with JWT

auth.mjs

```mjs
import jwt from 'jsonwebtoken';

// login: issue a signed token, no session stored
const token = jwt.sign(
  { sub: user.id, role: user.role },
  process.env.JWT_SECRET,
  { expiresIn: '1h' }
);

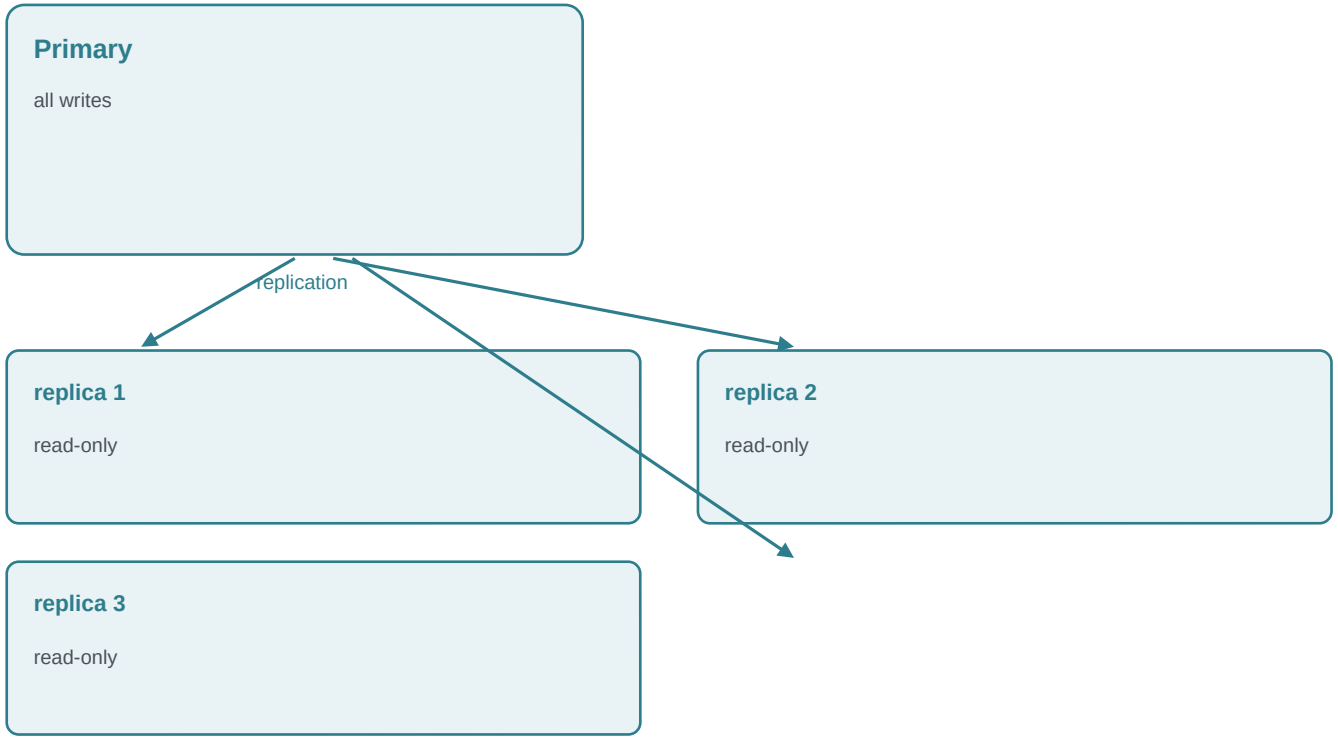
// every request: verify and trust, no DB hit
const payload = jwt.verify(token, process.env.JWT_SECRET);
```

Why JWT fits scaling

- The token carries the identity; the server stores nothing.
- Any node can verify it with the shared secret.
- No sticky sessions, no session store to share.
- Downside: hard to revoke a single token — keep TTL short.

Server-side sessions force sticky routing or a shared store. JWT moves the state to the client.

Scaling the database



What this buys

- Reads fan out to replicas — most apps are read-heavy.
- Writes still go to one primary (the real limit).
- Replication lag means replicas can be slightly stale.
- Sharding (splitting data across DBs) is the next step, and much harder.

Takeaways

01

Scale out, not up

Horizontal scaling is how the web scales, and it is the theme of the session.

02

Go stateless

No per-user state in memory. Sessions become JWTs; state lives in shared stores.

03

One loop per core

cluster gives you one event loop per core on a single machine.

04

LB + replicas

A load balancer fans out traffic; read replicas scale the database.

TOPIC 3 · 35 MIN

Caching Strategy Architecture

Add speed from the bottom up: app, Redis, reverse proxy, browser.

Why we cache

Latency

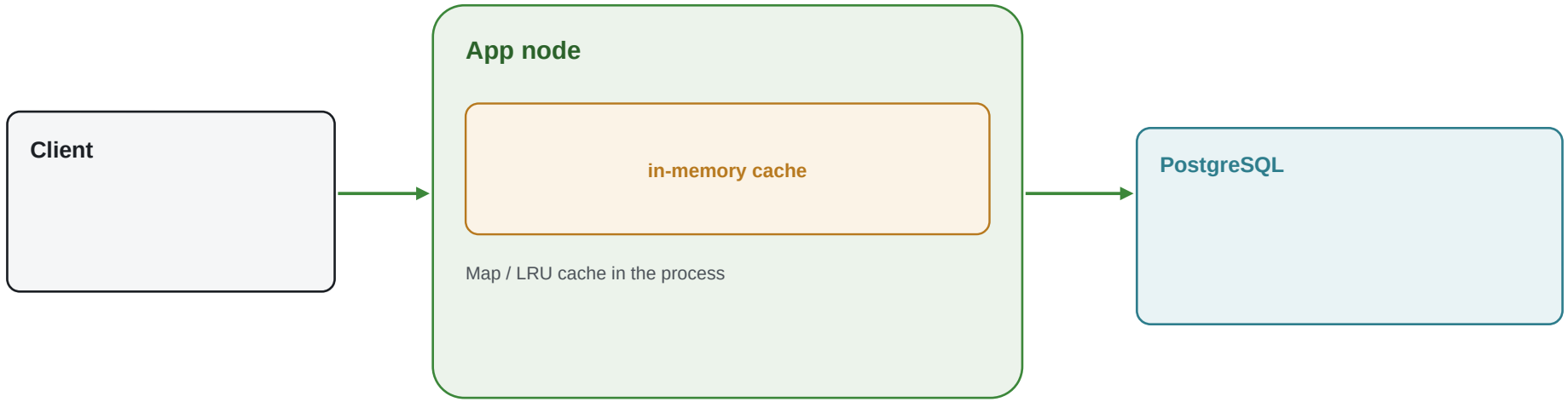
- Memory is ~100× faster than a DB query.
- One slow page loses users; caching cuts p99.

Load

- A cache hit never touches the database.
- Protects the DB during traffic spikes.

Caching trades staleness for speed and headroom. The whole art is deciding what may be stale, and for how long.

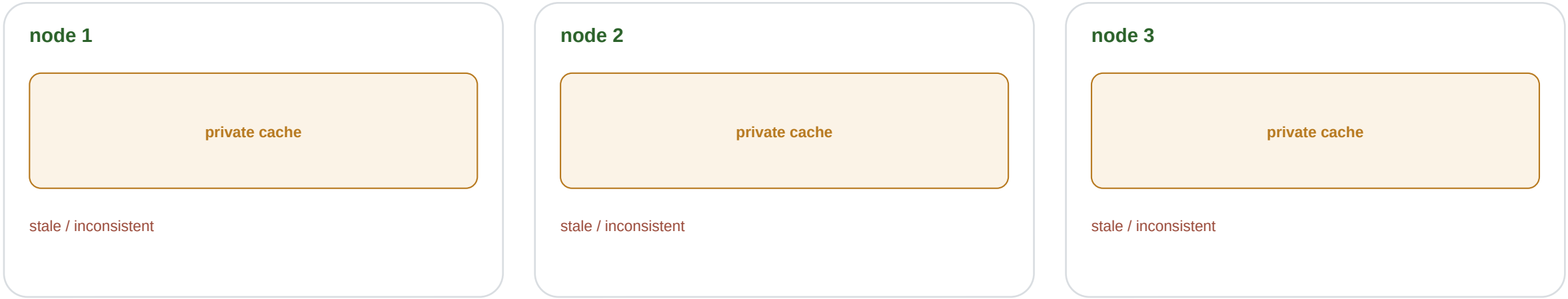
In-app cache (memory)



- Fastest possible cache — no network, no serialization.
- Fine while there is exactly one process.
- Breaks the moment you cluster or scale out — each node gets its own private cache.

Step 1 is correct for one process — and wrong for every system we scale.

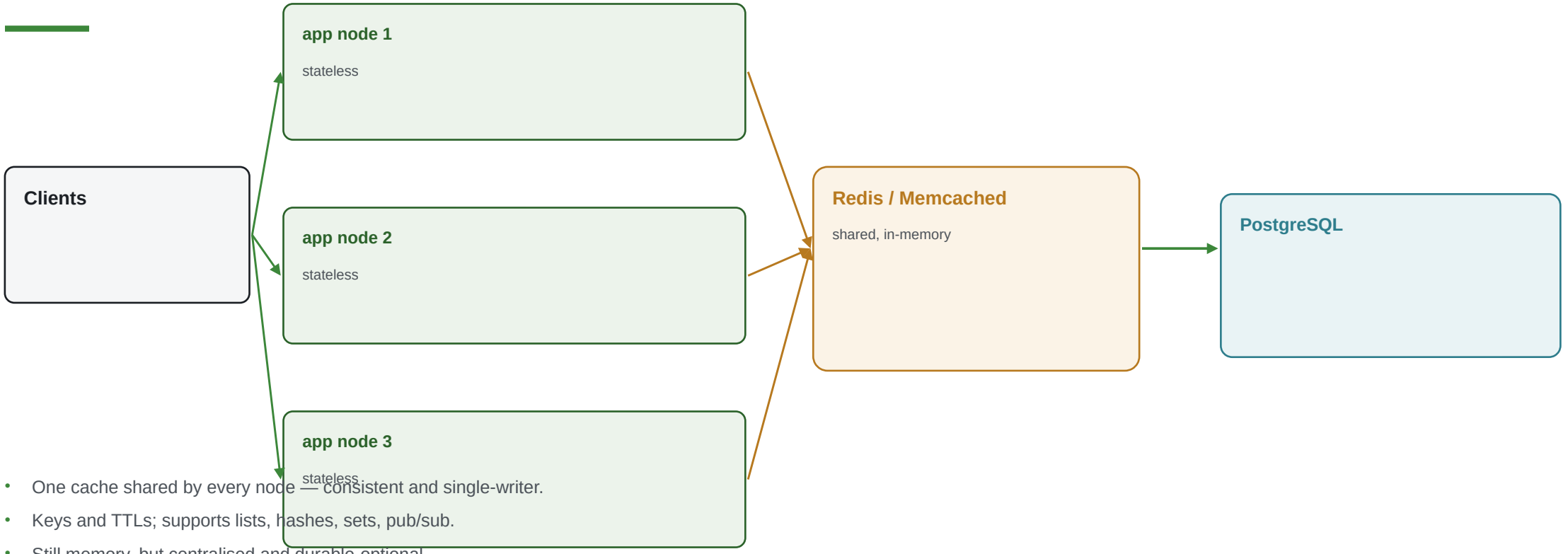
Why in-app cache fails when clustered



- Duplicate memory — the same data cached N times across N nodes.
- Inconsistency — node 1 sees fresh data, node 2 serves stale data.
- Cache misses multiply — each node warms its own cache from the DB.
- No shared invalidation — a write on one node cannot tell the others.

The fix: move the cache out of the process into a shared store.

Redis / Memcached: a shared cache



- One cache shared by every node — consistent and single-writer.
- Keys and TTLs; supports lists, hashes, sets, pub/sub.
- Still memory, but centralised and durable-optional.

Cache-aside

cache-aside.mjs

```
async function getUser(id) {
  // 1. try cache
  const hit = await redis.get(`user:${id}`);
  if (hit) return JSON.parse(hit);

  // 2. miss: load from DB
  const user = await db.query('SELECT * FROM users WHERE id=$1', [id]);

  // 3. store it for next time
  await redis.set(`user:${id}`, JSON.stringify(user), 'EX', 300);
  return user;
}
```

The pattern

Check cache → on miss, read DB → store in cache → return.

Lazy by design

Only requested data gets cached.

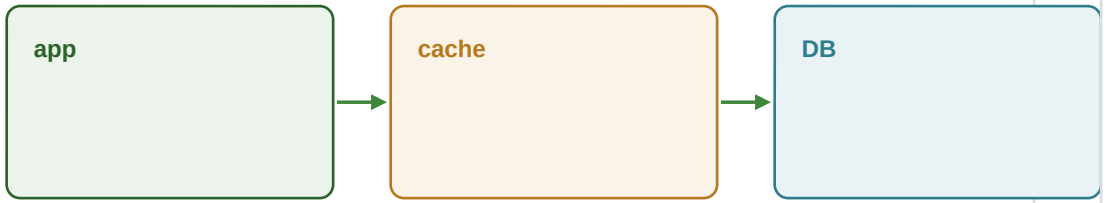
Trade-off

First request is slow; data may be stale until TTL or invalidation.

> **Cache-aside is the most common pattern: simple, lazy, and safe when data is read-heavy.**

Write-through vs write-around

Write-through



Write both together: cache stays fresh. Cost: every write touches both.

Write-around



Write only to DB; cache filled lazily on read. Avoids caching rarely-read writes.

- Write-through = always fresh, but writes cost more.
- Write-around = cheaper writes, cache misses on the next read until it is refilled.
- (Write-back: write to cache, flush to DB later — fastest but riskiest.)

Keeping the cache correct

“There are only two hard things in computer science: cache invalidation and naming things.”

TTL

Every key expires after N seconds. Simple, always converges, but may serve stale data briefly.

Explicit invalidation

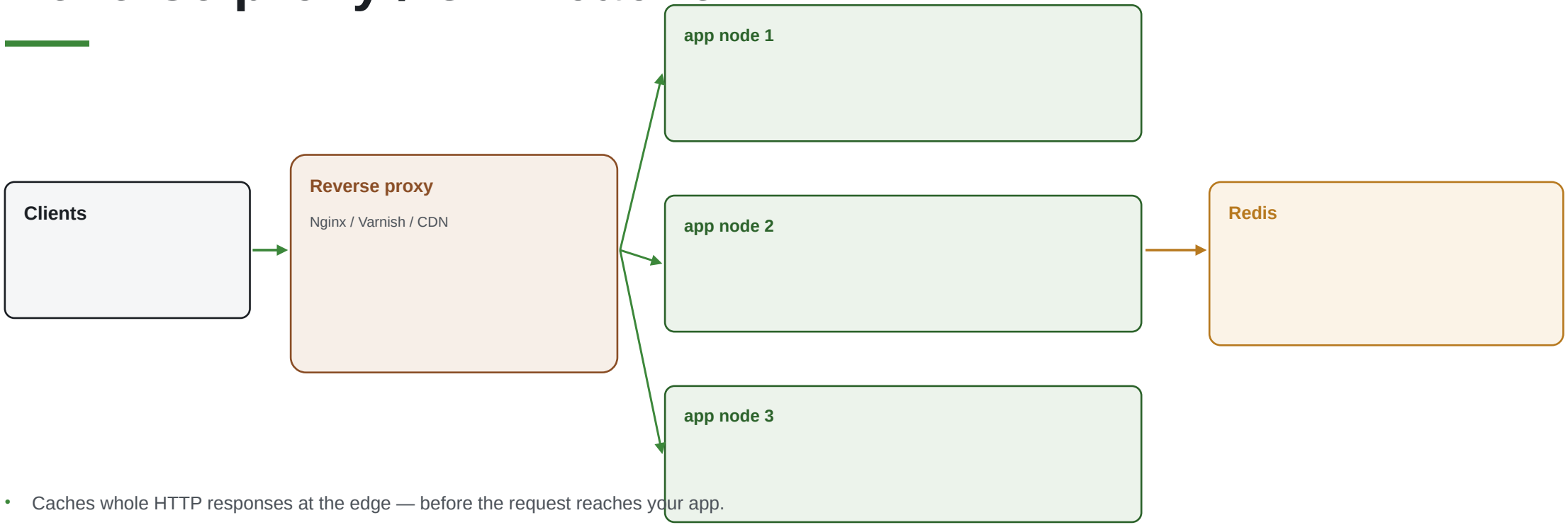
On write, delete or update the related key. Precise, but easy to miss a key.

Versioned keys

Key includes a version: user:42:v3. Never mutate; publish a new version.

Combination: a TTL as a safety net, plus explicit invalidation on writes. Never rely on memory — it must self-heal after a restart.

Reverse proxy / CDN cache



- Caches whole HTTP responses at the edge — before the request reaches your app.
- CDN puts it geographically near the user (static assets, GET responses).
- Controlled by Cache-Control and proxy-cache headers your app sets.
- Protects the app and DB from the largest, most cacheable share of traffic.

The browser cache

response.headers

```
// response from your API
HTTP/1.1 200 OK
Cache-Control: public, max-age=3600   // 1 hour
ETag: "v3"                            // revalidate cheaply
Vary: Accept-Encoding
```

max-age
how long the client may reuse the response without asking.

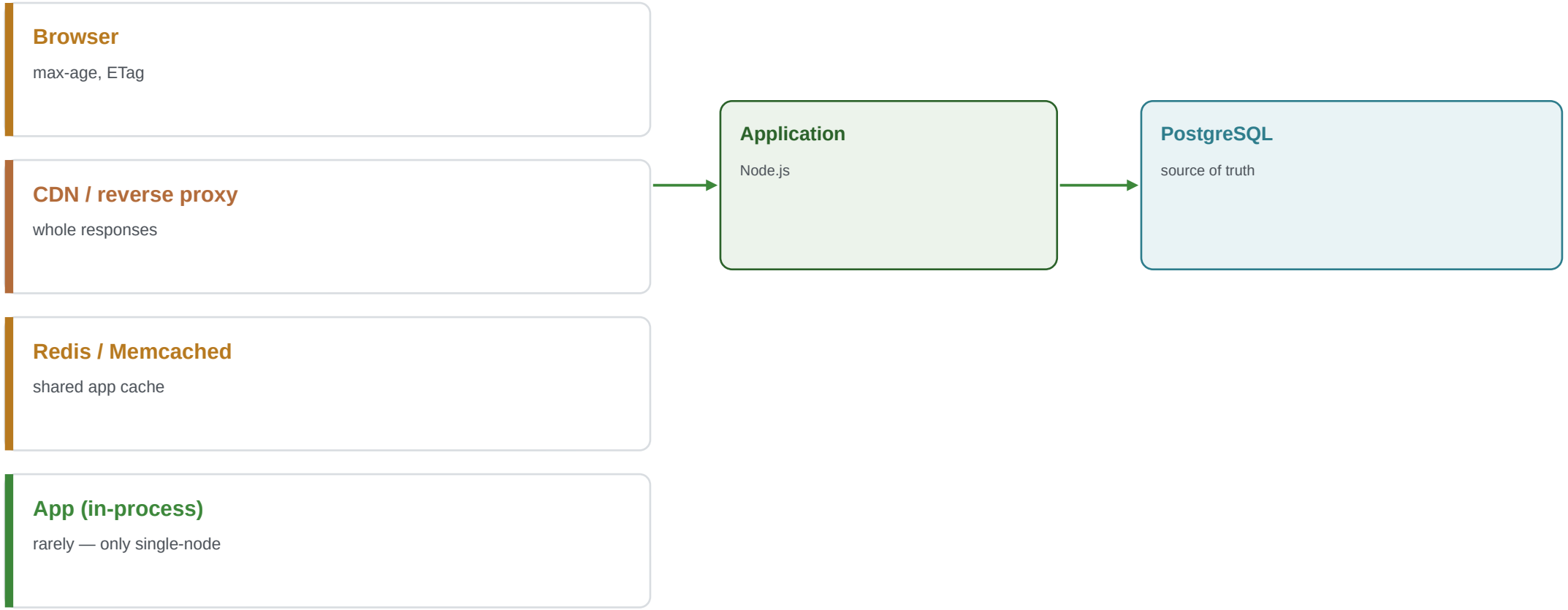
no-store / no-cache
never cache / always revalidate.

ETag
a version token — the client asks “still current?” and gets 304 Not Modified.

private vs public
who may cache it (one user vs shared proxies).

> **The cheapest cache of all: the client already has the data, so it does not even send a request. Control it with Cache-Control and ETag.**

All layers together



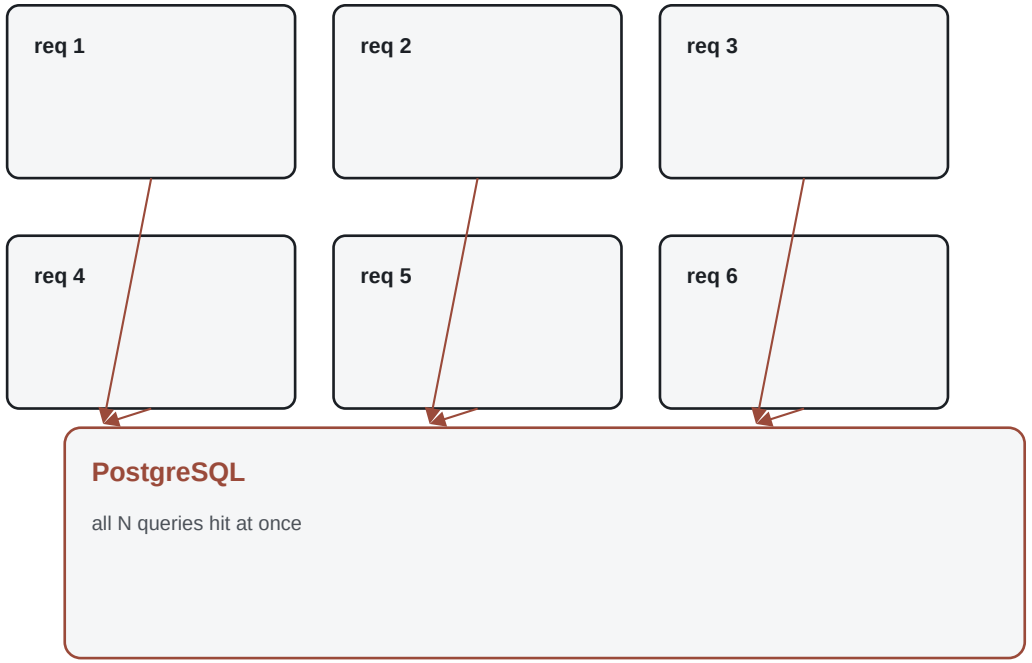
Each layer answers the query before the one behind it sees it. A miss falls through to the next.

Cache stampede (thundering herd)

A hot key expires; a thousand requests all miss at once and hammer the database.

hot key

expired



Fixes

Jitter the TTL

random expiry, so keys don't all die together

Lock on miss

one request rebuilds, others wait

Early refresh

refresh before expiry

Takeaways

01

Cache bottom-up

Browser → CDN/RP → Redis → in-process. Answer as far from the DB as possible.

02

Shared, not in-process

Once clustered, the cache must live in a shared store like Redis.

03

Choose read & write paths

Cache-aside to read; write-through / write-around to write.

04

Invalidate deliberately

TTL as a safety net + explicit invalidation. Beware the stampede.

TOPIC 4 · 25 MIN

Rate Limiting & API Protection

Guard the system from abuse, runaway clients, and cost spikes.

Why limit at all

Abuse & brute force

Slow down credential stuffing and scraping.

Fairness

One noisy tenant must not starve everyone else.

Cost control

Third-party APIs bill per call; runaway loops are expensive.

Availability

Rate limiting is a first line of defence against floods.

Rate limiting is not just security — it is how you keep the service usable and affordable under load.

Fixed window counter



- Simple: count requests in a fixed period; reject when the limit is hit.
- Problem: a burst at the boundary counts in two windows — up to 2× the limit slips through.
- Good enough for many cases; easy to explain and to store (a counter + a timestamp).

Sliding window & token bucket

Sliding window

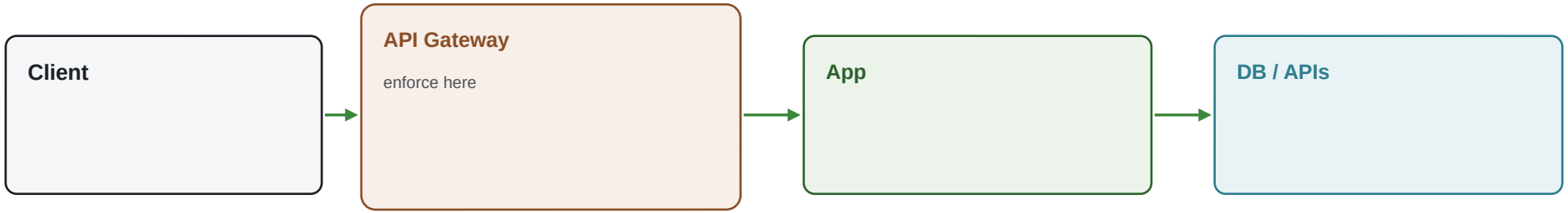
- Count only the last N seconds from now.
- No boundary burst — smooth and accurate.
- Needs a sorted set / log per key (Redis ZSET).

Token bucket

- A bucket refills at a steady rate; each request takes a token.
- Allows short bursts up to the bucket size.
- Good for smoothing traffic to a steady rate.

Pick by need: sliding window for strict per-key limits; token bucket when you want to allow bursts but enforce an average.

Where to enforce it



- Gateway / reverse proxy — one place, before traffic reaches the app. Best default.
- In-app middleware — fine-grained, per-route rules, but each node must share the counters.
- Upstream (outbound) — protect a third-party API you call, to stay under its quota.
- Distributed state is the hard part: counters must live in Redis, not in a node's memory.

Distributed limiting with Redis

ratelimit.mjs

```mjs
// fixed window per client
const key = `rl:${clientId}:${minute}`;

// atomic increment + set expiry once
const count = await redis.incr(key);
if (count === 1) await redis.expire(key, 60);

if (count > LIMIT) {
  return res.status(429).set('Retry-After', 60).end();
}
```

Redis is single-threaded

INCR is atomic, so two nodes can't double-count.

EXPIRE

the window length; set once on first hit.

429 + Retry-After

tell the client it is throttled and when to retry.

Headers

X-RateLimit-Limit / Remaining / Reset.

This is exactly the shared-store lesson from caching: one Redis, many app nodes, correct counters.

The rest of API protection

API keys

Identify who is calling. Rotate and revoke keys; never trust them alone.

Authentication

JWT or OAuth2 for real identity, on top of the key.

IP allow-lists

Block or allow by network — coarse but effective for admin endpoints.

WAF / DDoS

A web application firewall filters known attack patterns before your app.

Rate limiting is one layer. Identity, keys, network rules and a WAF together form real API protection.

Takeaways

01

It protects availability

Abuse, fairness, cost and stability — not just security.

02

Pick the algorithm

Fixed window is easy; sliding window is accurate; token bucket allows bursts.

03

Enforce at the edge

Gateway first; in-app for per-route; shared Redis counters always.

04

Signal clearly

429 + Retry-After + X-RateLimit-* so clients throttle themselves.

TOPIC 5 · 45 MIN

Background Jobs & Queues

Do slow work off the request path — asynchronously, reliably, retryably.

Long work must not block the loop

The anti-pattern

bad.mjs

```
app.post('/report', async (req, res) => {
  const data = await buildBigReport(req.body); // 30s
  await sendEmail(req.body.email, data);       // 10s
  res.json({ ok: true });
});
```

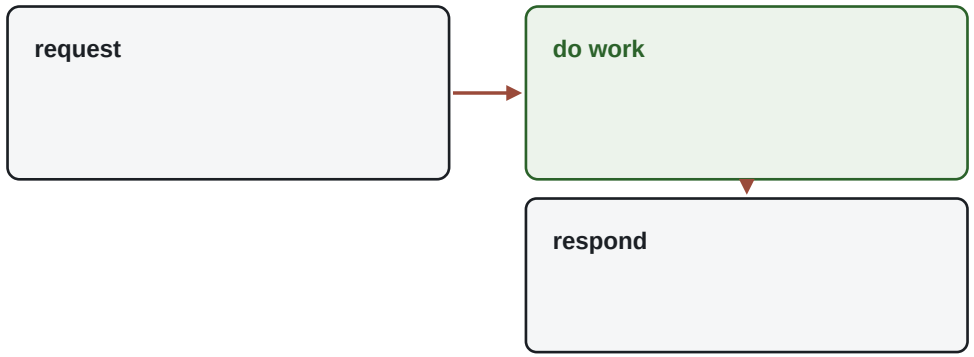
What breaks

- The user waits 40 seconds for a response.
- The request holds a connection — and often a DB row — the whole time.
- A crash mid-task loses the work with no retry.
- Spikes queue up in the app itself, exhausting it.

Anything slow, heavy or failure-prone belongs off the request path — in a queue.

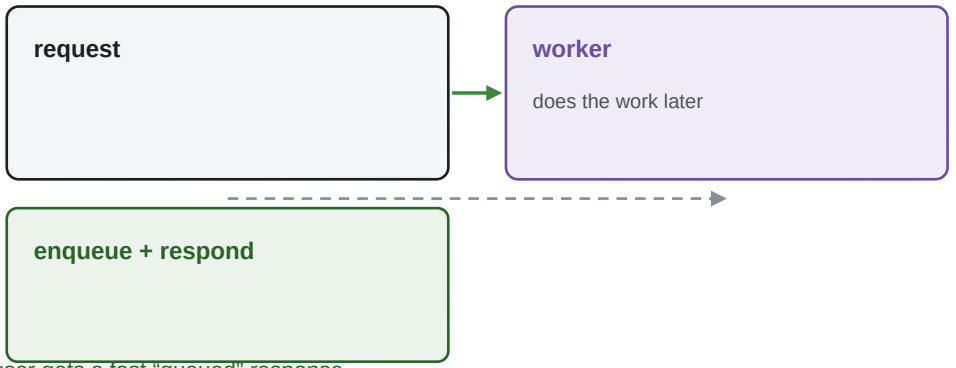
The shape of the change

Synchronous



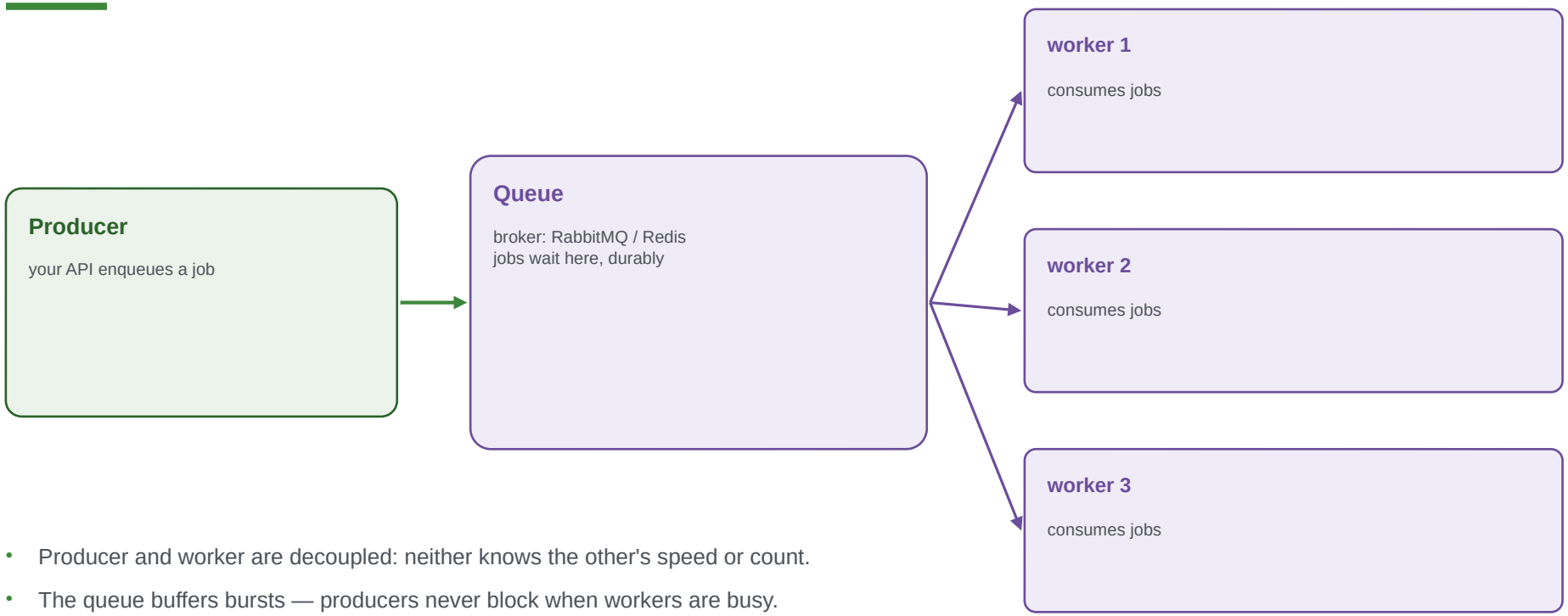
user waits for the whole thing

Asynchronous



user gets a fast “queued” response

Producer → queue → worker



- Producer and worker are decoupled: neither knows the other's speed or count.
- The queue buffers bursts — producers never block when workers are busy.
- Durable queue = the job survives a crash of the app or a worker.
- Scale by adding workers, independently of the API.

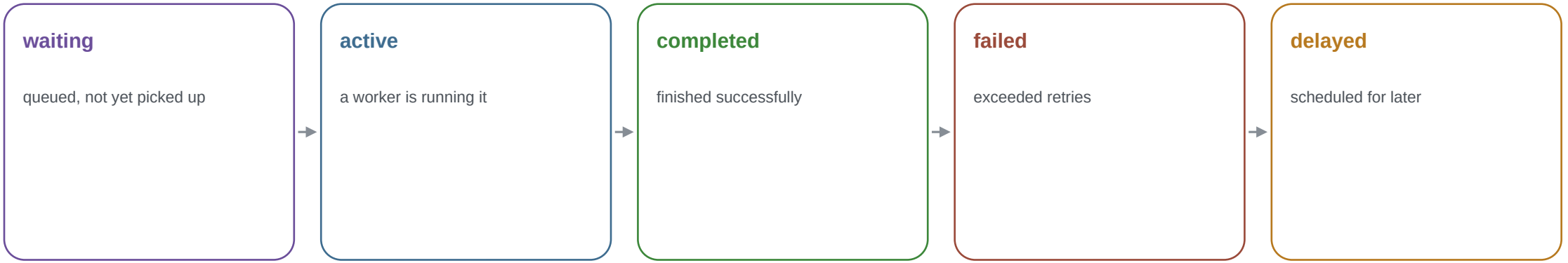
Message broker vs task queue

	Message broker	Task queue
Example	RabbitMQ, Kafka	BullMQ (Redis), Celery
Delivery	push to consumers	pull by workers
Durability	strong (disk)	depends on Redis
Ordering	guaranteed (queues)	best-effort
Best for	events, fan-out	job scheduling, retries

For background jobs in Node, BullMQ on Redis is the common default. For events between services, RabbitMQ or Kafka.

QUEUES · LIFECYCLE

A job's life



Every job is stateful: it moves through these states. Observing them is how you know the system is healthy.

BullMQ exposes these states as queues you can monitor: waiting, active, completed, failed, delayed.

Retries, backoff, idempotency

queue.mjs

```mjs
// BullMQ: retry with exponential backoff
const queue = new Queue('email');

await queue.add('send', { to, template }, {
  attempts: 5,
  backoff: { type: 'exponential', delay: 2000 },
  // idempotency key: re-adding the same id is a no-op
  jobId: `email:${userId}:${campaignId}`,
});
```

Retries

transient failures (network, DB down) should retry, not die.

Backoff

wait longer between attempts: 2s → 4s → 8s.

Idempotency

a retried job must not run twice — use a stable job id.

Dead-letter queue

after N failures, park the job for inspection.

The golden rule: make every job idempotent. Retries are inevitable; double-execution must be harmless.

The dead-letter queue (DLQ)



- After the configured attempts, the job moves to the DLQ instead of being retried forever.
- A growing DLQ is your alert: something is systematically failing.
- Replay the job after fixing the bug, or discard with a reason.

Delayed, scheduled & prioritised jobs

Delayed

Run once, N minutes from now — reminders, retry-later, rate-limit follow-ups.

Scheduled

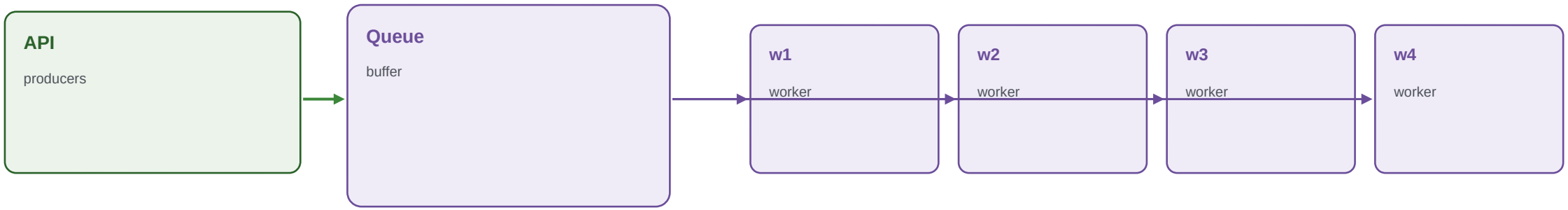
Cron-like recurring jobs — nightly reports, daily digest, cleanup.

Prioritised

A priority per queue, so urgent jobs jump ahead of bulk work.

The queue becomes your scheduler — no cron scripts, no timers in the app, one mechanism for all of it.

Scale workers independently



- Workers are just processes; add or remove them to match the backlog.
- The queue smooths the difference: producers never see a busy worker.
- CPU-heavy jobs → one per core; I/O-heavy jobs → many per process.
- This is the horizontal-scaling lesson again, applied to background work.

QUEUES · EXAMPLES

What belongs in a queue

Email & SMS

Send in the background; retry on provider errors.

Image / video processing

Resize, transcode, thumbnail — CPU-heavy, slow.

Reports & exports

Build a CSV/PDF and email a link when ready.

Webhooks

Deliver events to third parties with retries and backoff.

Search indexing

Reindex a document when it changes.

Data cleanup

Nightly deletes, expiry, reconciliation.

Rule: if it is slow, heavy, or failure-prone, it is a queue job — not an HTTP request.

Takeaways

01

Off the request path

Slow, heavy, failure-prone work goes to a queue, not the request.

02

Producer → queue → worker

Decouple, buffer bursts, scale each side independently.

03

Retries + idempotency

Back off between retries; a stable job id makes re-runs harmless.

04

DLQ as the safety net

Exhausted jobs park for inspection; watch its size.

WRAP-UP

The whole picture

Microservices	split only when the team needs it
Stateless + JWT	the prerequisite for scaling out
Caching	browser → CDN → Redis, bottom-up
Rate limiting	Redis counters at the edge
Queues	slow work off the request path

One story: take a two-tier app, make it stateless, cache it, protect it, and queue the slow work.